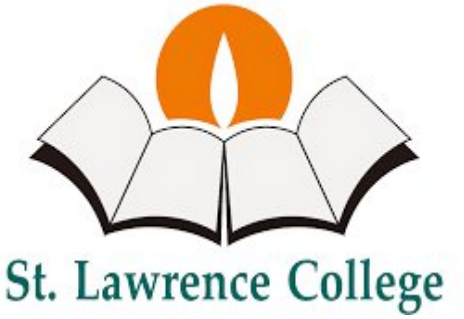
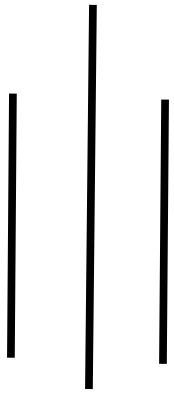
Report on Linux Case Study





Submitted By:

Shudarsan Paudel

BscCSIT 4th Sem

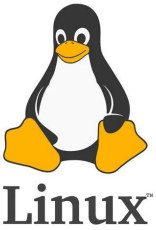
Submitted To:

Roshan Thapa

Table of Contents

History of Linux

Linux is one of the most influential operating systems in modern computing. Its history dates back to the early 1990s, when the need for a free and open-source alternative to commercial UNIX systems was growing.



1. **Origins (1969 – 1980s): UNIX as Inspiration**
    - The foundation of Linux comes from **UNIX**, developed at AT&T's Bell Labs in 1969 by **Ken Thompson and Dennis Ritchie**.
    - UNIX introduced key concepts such as multitasking, multi-user capabilities, and portability, which later inspired Linux.
    - During the 1980s, various UNIX variants emerged, but they were mostly proprietary and expensive.
2. **GNU Project (1983): Building Free Software**
    - In 1983, **Richard Stallman** launched the **GNU Project** with the goal of creating a completely free UNIX-like operating system.
    - The Free Software Foundation (FSF) was founded in 1985 to support this mission.
    - By the late 1980s, most components of GNU (compilers, libraries, editors, etc.) were developed, but the kernel (called GNU Hurd) was incomplete.
3. **Birth of Linux (1991): Linus Torvalds' Contribution**
    - In 1991, **Linus Torvalds**, a Finnish computer science student, started developing a personal project: a free kernel that could run on Intel's 386 processor.
    - On **August 25, 1991**, Torvalds announced his project on the Usenet newsgroup *comp.os.minix*, seeking contributions and feedback.
    - The first official version, **Linux 0.01**, was released in **September 1991**, followed by **Linux 1.0** in **March 1994**.
4. **Collaboration with GNU**
    - The Linux kernel combined with GNU software (tools, compilers, libraries) created a fully functional, free operating system.

- This collaboration is often called **GNU/Linux**, though most people simply refer to it as **Linux**.

5. **Rise of Linux Distributions (1990s – 2000s)**
    - Early Linux adopters packaged the kernel with software into **distributions** to make installation easier. Examples include:
        - **Slackware (1993)** – one of the earliest Linux distributions.
        - **Debian (1993)** – a community-driven distribution still widely used today.
        - **Red Hat Linux (1994)** – focused on enterprise use.
    - Over time, many other distributions emerged, such as **Ubuntu (2004)**, making Linux more user-friendly for desktops.
6. **Modern Linux (2000s – Present)**
    - Linux has become the backbone of modern computing. It powers:
        - **Servers** (majority of web servers run Linux).
        - **Supercomputers** (all of the world's top 500 supercomputers run Linux).
        - **Mobile devices** (Android is based on the Linux kernel).
        - **Embedded systems, IoT devices, and cloud infrastructure**.
    - The Linux kernel is actively developed by thousands of contributors worldwide, co-ordinated by Linus Torvalds and the Linux Foundation.

In summary: Linux began as a hobby project by Linus Torvalds in 1991 but grew, with the help of the GNU project and open-source community, into the most widely used and versatile operating system kernel in the world today.

Features of Linux Operating System

Linux is widely used across servers, desktops, mobile devices, and embedded systems because of its flexibility and open-source nature. The following are its key features:

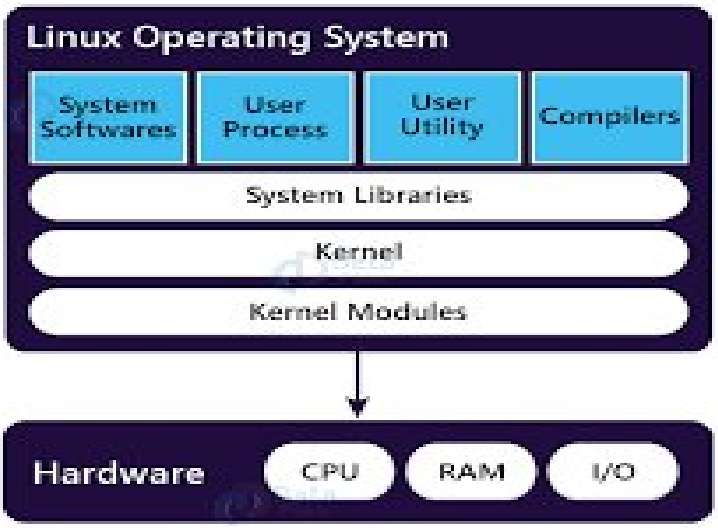
1. **Open Source and Free**
    - The source code of Linux is freely available to anyone.
    - Users can study, modify, and redistribute it without licensing costs.
    - Source Code URL

2. **Portability**
    - Linux can run on a wide variety of hardware platforms, from supercomputers to smartphones and IoT devices.
    - Its kernel is highly adaptable to different architectures.
3. **Multiuser Capability**
    - Multiple users can log in and use system resources (CPU, memory, storage) simultaneously without interfering with each other.
4. **Multitasking**
    - Linux efficiently handles multiple processes at the same time, making it suitable for both servers and desktop systems.
5. **Security and Permissions**
    - Linux uses a robust file permission and ownership system (read, write, execute).
    - Built-in tools like firewalls, SELinux, and encryption enhance system security.
6. **Stability and Reliability**
    - Linux is known for its stability; it can run for years without needing a reboot.
    - Widely used in mission-critical environments such as servers and scientific computing.
7. **Shell and Command Line Interface**
    - Linux provides powerful shells (e.g., Bash, Zsh) that allow users to control the system with commands and scripts.
8. **File System Support**
    - Supports multiple file systems such as ext4, XFS, Btrfs, FAT, NTFS, and more.
    - Provides hierarchical directory structure for efficient file organization.
9. **Networking Features**
    - Linux is designed with strong networking capabilities.
    - It is widely used in servers, routers, firewalls, and cloud environments.
10. **Community Support and Frequent Updates**
    - A large global community contributes to Linux development.
    - Continuous updates ensure security, performance improvements, and new features.
11. **Customizability**
    - Users can customize almost everything—from the desktop environment (GNOME, KDE, XFCE) to the kernel itself.
    - Lightweight versions exist for low-resource devices.

In summary: Linux stands out for being open-source, secure, stable, multitasking, and highly customizable, making it a preferred choice for personal, enterprise, and cloud computing.

Components of the Linux System

The Linux operating system is made up of several core components that work together to provide functionality. The major components are:



1. Kernel

- The core of the Linux system.
- Manages hardware resources (CPU, memory, devices, storage).
- Provides an interface between hardware and software.
- Types of kernels in Linux: **Monolithic Kernel** (Linux uses this), **Microkernel**, **Hybrid**.

2. System Libraries

- Special programs that applications use to access kernel features.
- Provide standard functions (e.g., handling input/output, memory, and process management).
- Example: **GNU C Library (glibc)**.

3. System Utilities

- Basic programs that perform essential system tasks.
- Examples:
    - **cp, mv, rm** → for file handling.
    - **ps, top, kill** → for process management.
    - **ifconfig, ping** → for networking.

4. Shell

- A command-line interpreter that acts as an interface between the user and the kernel.
- Takes commands from the user, executes them, and shows results.
- Examples: **Bash, Zsh, Fish, KornShell**.

5. Hardware Layer

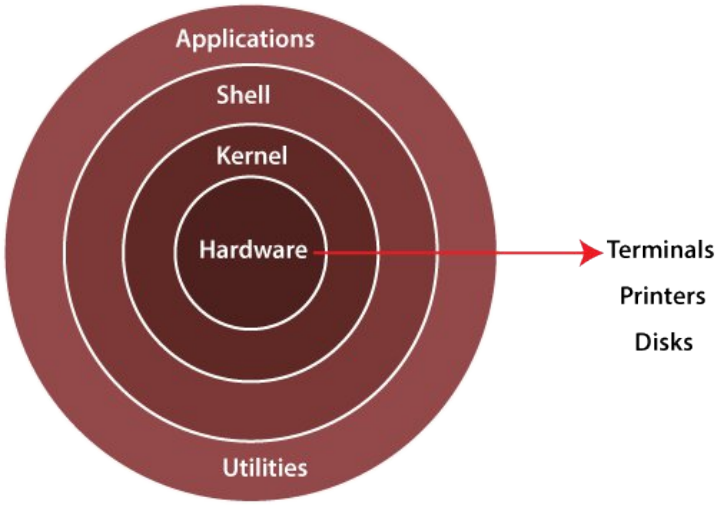
- The physical components of the computer (**CPU, RAM, hard disk, network card**, etc.).
- The kernel directly communicates with the hardware using **device drivers**.

6. Application Programs

- User-level programs that run on top of the system.
- Examples: text editors (**Vim, Nano**), browsers (**Firefox, Chrome**), databases (**MySQL, PostgreSQL**).
- These programs make Linux useful for end-users.

Linux System Architecture

Linux uses a **layered architecture** where each layer is designed to perform specific tasks and interacts with the other layers in a structured way. Unlike components (which describe *what Linux is made of*), architecture explains *how those components are organized and work together*.



1. User Space

This is the **top layer**, where users and applications interact with the system. It has two main parts:

- **User Applications**
    - Programs like text editors, browsers, media players, and databases.

    - These run in user mode and cannot directly access hardware.
- **System Utilities**
    - Provide essential commands and tools to manage the system.
    - Example: `cp`, `mv`, `ps`, `ping`.

---

2. Shell

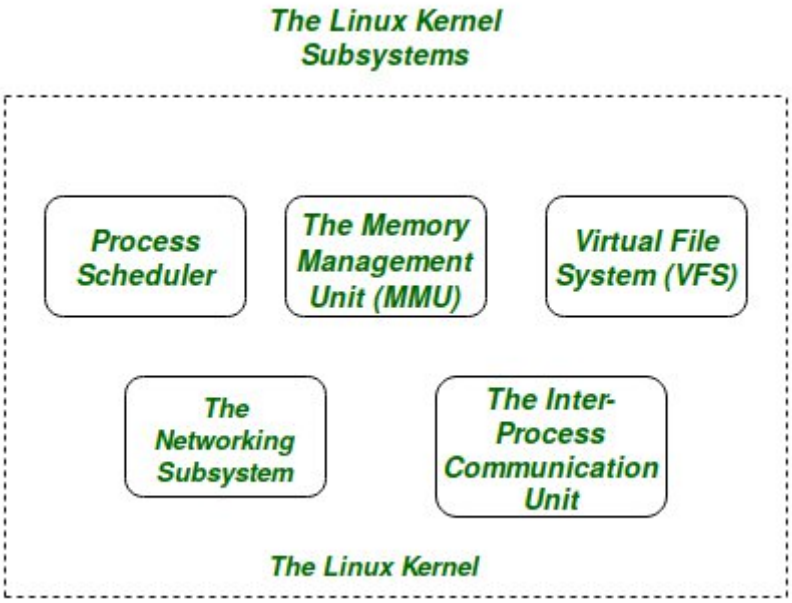
- Acts as the **interface between the user and the kernel**.
- Converts user commands (or scripts) into instructions for the operating system.
- Can be command-line based (Bash, Zsh) or graphical (GNOME Shell, KDE Plasma).

---

3. Kernel Space

This is the **core of the operating system** that runs in kernel mode with full access to hardware. It is divided into several subsystems:

- **Process Management** – scheduling, creating, and terminating processes.
- **Memory Management** – handling RAM allocation and virtual memory.
- **File System Management** – organizing and controlling access to data storage.
- **Device Drivers** – enabling communication between hardware devices and software.
- **Networking** – providing communication across systems via protocols.



---

4. Hardware Layer

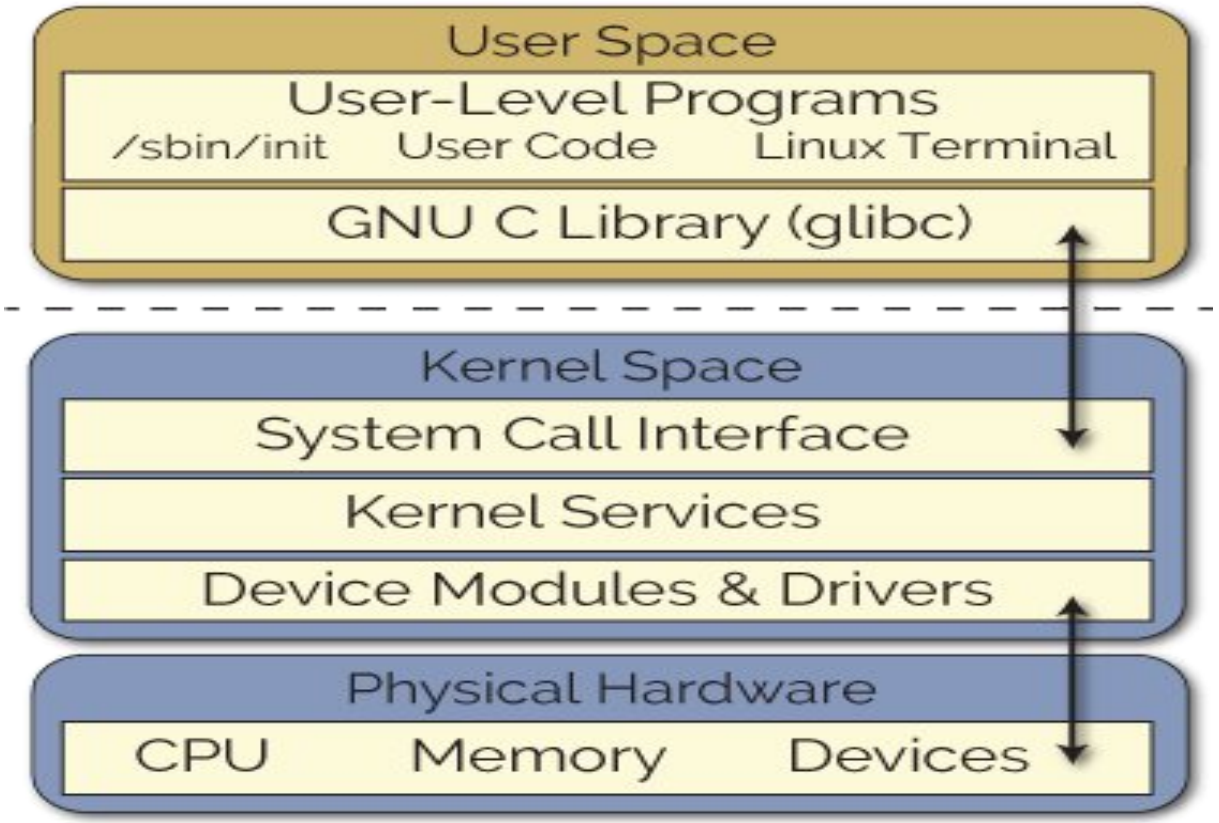
- The bottom layer of the Linux system.
- Includes CPU, memory, hard disks, I/O devices, and network interfaces.
- The kernel directly manages and controls these resources.

Linux Kernel Modules

Linux kernel modules are pieces of code that can be dynamically loaded into or unloaded from the Linux kernel at runtime, allowing the kernel to be extended without the need for rebooting the system. They offer a modular way to add functionality such as device drivers, file system support, system calls, or security features independently from the core kernel. This modular approach makes the kernel flexible and adaptable to various hardware and software environments.

What Are Linux Kernel Modules?

Kernel modules are dynamically loadable components that extend the functionalities of the monolithic Linux kernel. They can be inserted into the kernel to add new features or support new hardware devices and later removed without affecting the running system. This eliminates the need to recompile or reboot the kernel for new capabilities.



Uses of Kernel Modules

- **Device Drivers:** Most commonly used to add support for new hardware devices like keyboards, network cards, and storage devices.
- **File Systems:** Certain file systems can be implemented as modules, enabling dynamic loading and unloading.
- **System Calls:** They can introduce or modify system calls to extend kernel functionality.
- **Security:** Used to implement enhanced security features and access control mechanisms.

How to Work with Kernel Modules

- **Creating Modules:** Write in C, including initialization and cleanup functions.
- **Compiling:** Use kernel build tools to compile the module into a loadable object.
- **Loading & Unloading:** Use commands like `insmod` to insert a module and `rmmod` to remove it.
- **Managing Dependencies:** Tools like `modprobe` handle loading modules along with their dependencies.
- **Persistency:** Modules can be configured to load automatically at boot time via configuration files.

Linux Process Management Overview

- A **process** in Linux is an executing instance of a program identified by a Process ID (PID).
- Processes can be **foreground** (interactive, user-driven) or **background** (running silently without user input).
- Each process goes through states such as **Running**, **Sleeping** (waiting for resources), **Stopped**, **Zombie** (completed but waiting for parent to acknowledge), and **Orphan** (parent process terminated).

Process Scheduling and Priority

- Linux uses a **time-sharing** scheduler where CPU time is divided among processes based on their priority.
- Processes are divided into **real-time** (priorities 0-99) and **ordinary** (priorities 100-139) scheduled by different algorithms.
- The **nice** value (-20 to 19) lets users adjust the scheduling priority of processes; lower nice value means higher priority.

Key Process Management Commands

- `ps` - Display running processes.

- `top`/`htop` - Real-time view of processes and resource usage.
- `kill`, `killall`, `pkill` - Send signals to terminate processes by PID or name.
- `nice`, `renice` - Adjust process priority.
- `fg`, `bg` - Move processes between foreground and background.
- `pidof`, `pgrep` - Find process IDs by process name.
- `jobs` - List background jobs in the current shell.
- `nohup` - Run processes immune to hangups.

Process Lifecycle in Linux

- Processes are created, scheduled for CPU time, may sleep waiting for I/O or signals, and eventually terminate releasing their resources.
- The kernel handles **context switching** to save and restore process states.
- Orphan processes are adopted by the `init` process, and zombies are cleaned up to free resources.

Creation of Processes in Linux

In Linux, process creation typically happens through system calls that enable a running process to create a new process. The main system call for creating a process is `fork()`.

Key Points on Process Creation in Linux

- The `fork()` system call creates a new process by duplicating the calling (parent) process. The new process is called the child process.
- After `fork()`, both parent and child processes run independently. The child inherits a copy of the parent's memory, file descriptors, and execution context but has a unique Process ID (PID).
- The child process can then use the `exec()` system call to replace its memory space with a new program if needed, allowing different programs to be run.
- This creation forms a parent-child hierarchy with each process having a Parent Process ID (PPID) referencing its creator.
- The very first process created by the kernel at boot is `init` (or `systemd` on modern systems), which has PID 1 and is the ancestor of all other processes.

Process Creation Workflow

1. A process calls `fork()`.
2. The kernel creates a new task structure for the child.

3. The child process is a near-exact copy of the parent at the time of `fork()`.
4. The child gets a unique PID.
5. The child can run the same program or call `exec()` to load a different program.
6. Eventually, the child process terminates, and its resources are reclaimed.

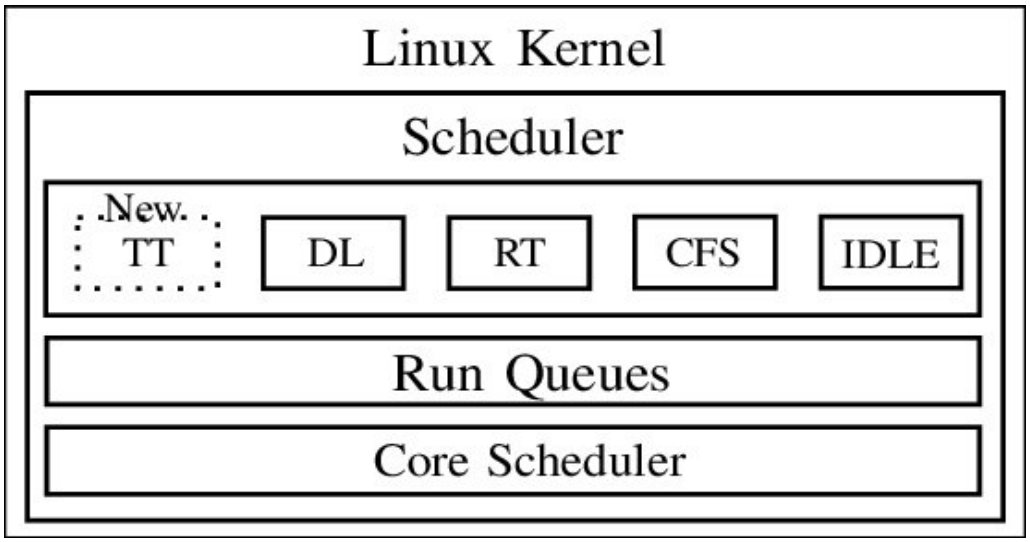
Example

- In a shell environment, when running a command, the shell uses `fork()` to create a child process and then `exec()` to run the command in that child process while the parent shell waits or performs other tasks.

This model provides a simple and efficient way to manage multiple concurrent processes in Linux.

Scheduler

The scheduler in Linux is a core component of the kernel responsible for managing and distributing CPU time among various processes and threads. Its primary goal is to ensure efficient utilization of the CPU while providing responsiveness and fairness to all running tasks.



Types of Scheduling in Linux

Linux supports several scheduling policies depending on the workload:

1. Normal (Time-Sharing) Scheduling

- Default for most processes.
- Managed by the **Completely Fair Scheduler (CFS)** since kernel 2.6.23.
- Goal: Fairly share CPU among processes.
- Each process gets a portion of CPU time (based on priority & weight).

2. Real-Time Scheduling

- Used for tasks that need **deterministic timing** (e.g., multimedia, robotics).
- Two main policies:
    - **SCHED_FIFO**: First-In, First-Out queue. Higher-priority processes run until they block or finish.
    - **SCHED_RR**: Round Robin. Like FIFO but with fixed time slices to prevent starvation.

3. Deadline Scheduling (SCHED_DEADLINE)

- Designed for tasks that must complete by a certain time.
- Uses parameters:
    - **Runtime** (CPU time needed),
    - **Deadline** (by when it must finish),
    - **Period** (how often it repeats).

Linux Scheduler Classes

1. **CFS (Completely Fair Scheduler)**
    - Default scheduler for normal tasks.
    - Shares CPU time fairly among processes using a *virtual runtime* system.
2. **RT (Real-Time Scheduling)**
    - Used for time-critical tasks.
    - Policies:
        - **SCHED_FIFO (First-In-First-Out):** Highest priority runs until it blocks/finishes.
        - **SCHED_RR (Round Robin):** Like FIFO but with fixed time slices.
3. **DL (SCHED_DEADLINE)**
    - For tasks with strict timing requirements.
    - Each task defines **runtime, deadline, and period**.
    - Ensures tasks finish before their deadlines.
4. **Idle (SCHED_IDLE)**
    - Lowest-priority policy.
    - Runs only when no other runnable task exists.

- Useful for background/low-importance jobs.

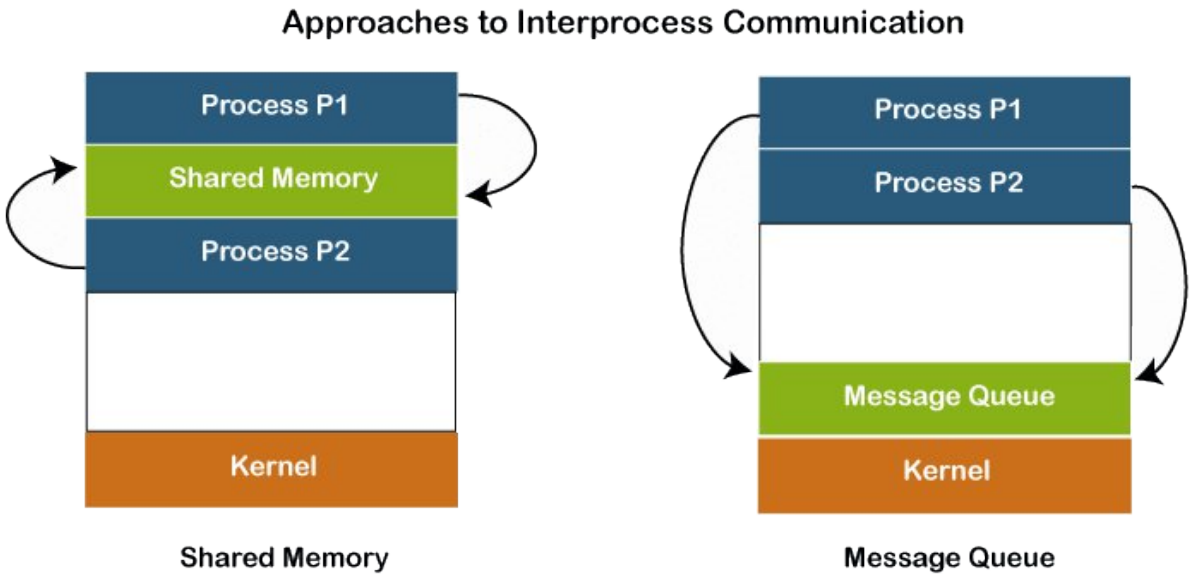
5. **TT (likely refers to “Throughput-oriented tasks” or batch scheduling in some texts)**
    - In Linux, this usually maps to **SCHED_BATCH** or **SCHED_IDLE** classes.
    - Designed for non-interactive, long-running jobs where throughput matters more than interactivity.

In short:

- **CFS → normal tasks**
- **RT → real-time tasks (FIFO/RR)**
- **DL → deadline tasks**
- **Idle → background jobs**
- **TT → batch/throughput tasks**

Inter-Process Communication (IPC) in Linux

Inter-Process Communication (IPC) is a mechanism that allows processes to exchange data and synchronize their actions. Since processes in Linux run in isolated memory spaces, IPC provides controlled ways to share information.



Main IPC Mechanisms in Linux

1. **Pipes and FIFOs** – Unidirectional or bidirectional channels for data transfer between related or unrelated processes.
2. **Message Queues** – Allow processes to send and receive structured messages in a queue.
3. **Shared Memory** – Fastest IPC method; processes share a common memory segment for direct data exchange (needs synchronization tools like semaphores).
4. **Semaphores** – Used to control access to shared resources and prevent race conditions.
5. **Signals** – Lightweight way to notify a process about events (e.g., `SIGINT`, `SIGKILL`).
6. **Sockets** – Provide communication between processes on the same or different systems (used in client-server applications).

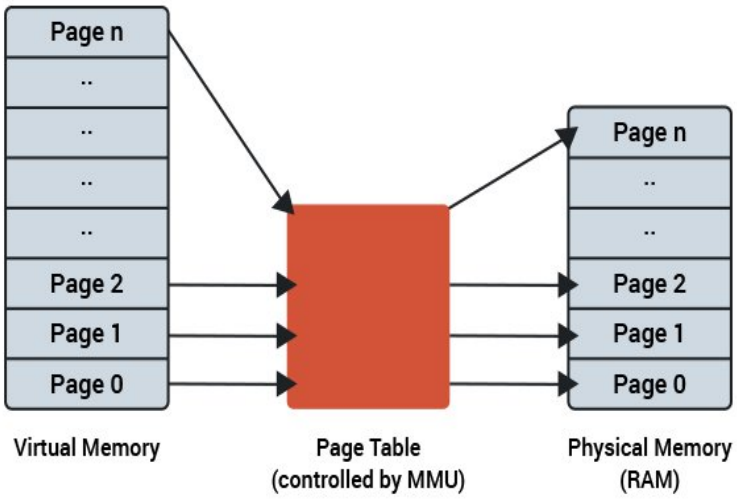
Memory Management in Linux

Introduction

Memory management in Linux is the process by which the kernel **allocates, tracks, and manages system memory** for processes efficiently. It ensures **protection, sharing, and optimal utilization** of RAM and provides support for both **user-space and kernel-space memory**.

Key Components of Linux Memory Management

1. **Virtual Memory**
    - Each process gets its own **virtual address space**, isolated from others.
    - Linux uses **paging** to map virtual addresses to physical memory.
    - Supports **demand paging**, **swapping**, and **copy-on-write**.



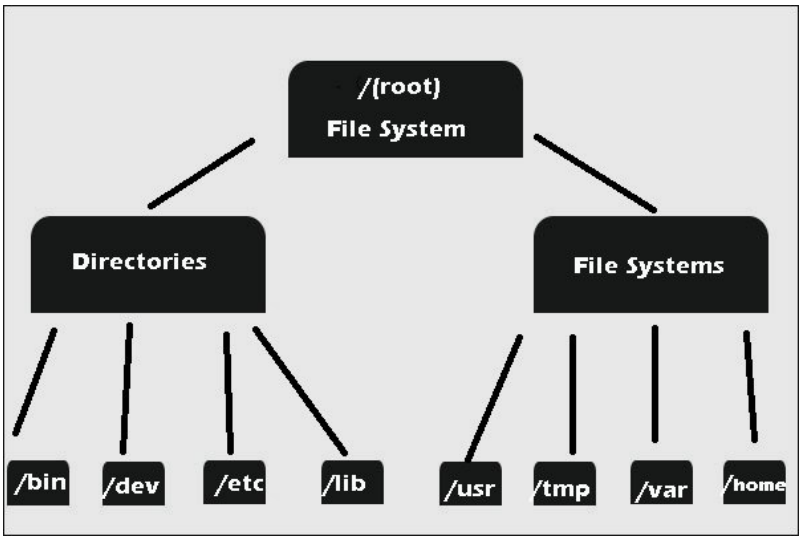
2. **Physical Memory Management**
    - Kernel manages **RAM as pages** (usually 4 KB each).
    - Keeps track of **free and used pages** using page tables and data structures like **buddy allocator**.

3. **Process Memory Layout**
    - Memory is divided into segments:
        - **Text Segment:** Executable code.
        - **Data Segment:** Initialized global/static variables.
        - **BSS Segment:** Uninitialized global/static variables.
        - **Heap:** Dynamic memory allocation (malloc).
        - **Stack:** Function call frames, local variables.
4. **Slab Allocator**
    - Kernel uses **slab allocator** for efficient allocation of small kernel objects.
    - Reduces fragmentation and improves performance.
5. **Swap Space**
    - Used when physical RAM is insufficient.
    - Inactive pages are moved to **swap space on disk**, freeing RAM for active processes.
6. **Memory Protection**
    - Ensures processes **cannot access each other's memory**.

- Uses **page-level protection** (read, write, execute).

File System Management in Linux

Linux file system management involves organizing, accessing, and manipulating files and directories within the operating system. This encompasses a hierarchical structure, file and directory permissions, and various commands for interaction



Linux File System Hierarchy (Directory Structure)

Linux organizes files into a **single-rooted tree** structure. Each directory has a specific purpose.

Key Directories:

Directory	Purpose
`/`	Root directory — top of the hierarchy. All directories stem from here.
`/bin`	Essential **binary executables** needed for all users (e.g., `ls`, `cp`, `mv`).
`/sbin`	System binaries — commands for **system administration** (e.g., `fsck`, `ifconfig`). Usually for root.
`/usr`	**User programs and data**. Contains most software and libraries. Subdirectories: - `/usr/bin` → non-essential binaries - `/usr/sbin` → non-essential system binaries - `/usr/lib` → libraries - `/usr/share` → architecture-independent data
`/lib`	**Shared libraries** for `/bin` and `/sbin`.

Directory	Purpose
`/etc`	System **configuration files** (text-based) like `passwd`, `fstab`.
`/home`	User **home directories** (e.g., `/home/shawn`).
`/root`	Home directory of **root user**.
`/var`	**Variable files**, like logs (`/var/log`), mail, databases.
`/tmp`	**Temporary files**, cleared on reboot.
`/dev`	Device files (hardware devices exposed as files, e.g., `/dev/sda`).
`/proc`	**Virtual filesystem** exposing kernel and process info.
`/sys`	Kernel and system device info (similar to `/proc`).
`/boot`	Boot loader files, kernel images.
`/opt`	Optional software packages.
`/mnt`	Temporary **mount points** for storage devices.
`/media`	Mount points for **removable media** (USB, CD).

Basic File System Commands in Linux

Command	Description	Example
`ls`	List files and directories	`ls`, `ls -l`, `ls -a`
`cd`	Change directory	`cd /home/user`, `cd ..`
`pwd`	Show current directory path	`pwd`
`cat`	View file contents	`cat file.txt`
`touch`	Create a new empty file	`touch newfile.txt`
`cp`	Copy files or directories	`cp file.txt /tmp/`, `cp -r folder1/ folder2/`
`mv`	Move or rename files	`mv file.txt /tmp/`, `mv old.txt new.txt`
`rm`	Remove files or directories	`rm file.txt`, `rm -r folder/`
`file`	Display the file type	`file file.txt`
`grep`	Search text inside files	`grep "hello" file.txt`

Device Management in Linux

Linux treats **hardware devices** (like disks, printers, USB drives, and network cards) as files. Device management involves **detecting, configuring, and controlling these devices** using the kernel and associated utilities.

---

1. Device Types

Linux classifies devices into **two main types**:

Device Type	Description	Examples
Character devices	Data is accessed **byte by byte**. Usually sequential access.	Keyboard, mouse, serial port (`/dev/ttyS0`)
Block devices	Data is accessed in **blocks** (fixed-size chunks). Supports random access.	Hard disks, SSDs (`/dev/sda`), USB drives

2. Device Files

- Linux uses the `/dev` **directory** to represent devices as files.
- **Naming conventions:**
    - `/dev/sda` → First SATA/SCSI disk

    - `/dev/sdb1` → First partition on second disk
    - `/dev/tty` → Terminal devices
- Device files allow **programs to read/write to devices** using standard system calls.

---

3. Device Drivers

- A **device driver** is software that communicates between the kernel and the hardware device.
- Responsibilities:
    - Translate generic I/O calls into hardware-specific operations.
    - Handle interrupts and direct memory access (DMA).
- Linux supports **modular drivers** via **kernel modules** (`.ko` files) that can be loaded/unloaded dynamically.

Command to manage drivers:

lsmod # List loaded modules

modprobe # Load/unload a kernel module

rmmod # Remove a module

4. Device Management Tools

Tool / Command	Purpose
`dmesg`	View kernel messages, especially device detection during boot
`lspci`	List all PCI devices
`lsusb`	List all USB devices
`fdisk -l`	List disks and partitions
`blkid`	Show UUIDs and file system types
`mount`	Mount a filesystem to access a device
`umount`	Unmount a device
`udevadm`	Manage device events handled by **udev**